

dotAI

A control layer for enterprise AI

An assistant answers a request. dotAI decides what the request actually requires, which model is capable of doing it, what information may safely reach that model, whether the answer can be trusted — and what should happen next.

ACCENTURE INNOVATION CHALLENGE · ROUND 2

Track 1 — ControlPlane.ai

Working prototype · 694 automated tests passing

Contents

- 1 Project overview
- 2 The problem ControlPlane.ai poses
- 3 Why existing approaches are insufficient
- 4 What makes dotAI different
- 5 System architecture
- 6 CAI — query understanding
- 7 Model Intelligence
- 8 The primary mini-task system
- 9 The curated model dataset
- 10 ControlPlane governance
- 11 Pre-generation privacy firewall
- 12 Action Gate — governing consequential actions **NEW**
- 13 Policy RAG and evidence
- 14 Governance profiles
- 15 Feedback, metrics and thresholds
- 16 Auditability and the decision trace
- 17 Multimodal chat
- 18 End-to-end worked example
- 19 Key features
- 20 Technology stack
- 21 Installation
- 22 Configuration
- 23 How to use dotAI
- 24 Example requests
- 25 Repository structure
- 26 Testing and verification
- 27 Security and secrets
- 28 Scope and limitations
- 29 Roadmap
- 30 Summary

HOW TO READ THIS DOCUMENT

Sections 1–5 explain what dotAI is and why it exists — no technical knowledge assumed. Sections 6–18 explain how each part works and which enterprise problem it solves. Sections 20–27 are for a developer setting the project up. Section 28 states plainly what is and is not implemented.

1 · Project overview

What dotAI is. A control layer that sits between an enterprise's users and the AI models it uses. It is not a chatbot and not a model. It is the thing that decides, for every individual request, which model should handle it, what information that model is allowed to receive, and whether the answer it produces may be delivered.

Who it is for. An organisation running AI across several workflows — customer support, internal knowledge, analysis feeding decisions — where those workflows have genuinely different tolerances for risk, cost and delay, and where somebody is accountable for what the AI does.

The problem it solves. Enterprises today face two failures that are usually treated as separate engineering problems. The first is economic: sending every request to the strongest available model is unjustifiably expensive, but routing everything to a cheap model produces weak answers on work that matters. The second is a governance failure: a model can answer confidently without grounds, and private data sent to an external provider cannot be recalled. dotAI treats these as one problem, because they are decided at the same moment — the moment a request is about to be executed.

What makes it different. Model selection and AI governance are normally built by different teams and never exchange a signal. In dotAI they are a single request-level path: what the request requires determines which models are eligible, and what governance applies determines what may be sent and what may be returned.

THE CENTRAL IDEA

An assistant answers a request. A control layer decides what is allowed to happen.

2 · The problem ControlPlane.ai poses

The Round-2 brief describes an enterprise running many AI models across many workflows, and asks for a control plane that governs them. The table below sets each challenge the brief raises against dotAI's response. Section numbers point to the implementation.

WHAT THE BRIEF STATES	HOW DOTAI ADDRESSES IT
Different use cases carry different risk tolerance and latency budgets	Four governance profiles with genuinely different control sequences — not four sets of thresholds (§14)
Overlapping risks, such as hallucination co-occurring with privacy exposure	Multi-label risk detection with intersection escalation: two co-occurring findings escalate together rather than counting as two mild flags (§10)
No reliable real-time ground truth	UNVERIFIABLE is a decision state, not a low confidence score. A claim with no evidence is escalated rather than assigned a number it never earned (§10)
Over-flagging vs under-flagging	An operating threshold chosen from a labelled set with false positives and negatives measured, and the reasoning recorded (§15)
Compounding risk across multi-turn interactions	Session risk accumulates across a conversation and raises verification depth as it climbs (§10)
Changing regulatory and policy expectations	Policy retrieval reads the organisation's own uploaded documents, so changing the document changes the decisions (§13)
API-based foundation models limit internal inspection	dotAI governs at the boundary — what is sent, and what is returned — because model internals are not available (§10, §11)
Need for detection, decision logic, architecture, governance, feedback and metrics	Each is a discrete implemented layer with its own tests (§5, §10, §15, §16)

WHY THE PROBLEM IS HARD

The difficulty is not that any single control is hard to build. It is that the correct decision depends on context that no single control has. Whether "send the customer their account balance" is routine or a reportable incident depends on the workflow it arrives in. A filter that inspects the answer cannot know that. A router that picks a cheap model cannot know it either. The context exists only at the request level, which is where dotAI operates.

3 · Why existing approaches are insufficient

Each approach below is useful and solves a real problem. None of them, alone, answers the brief.

APPROACH	WHAT IT SOLVES	WHAT IT DOES NOT SOLVE
Fixed model	Predictable behaviour and simple operations	Pays frontier prices for trivial work, and still lacks capabilities some requests need. Capability and cost differ per request.
Generic safety filter	Catches unsafe output before it reaches a user	Scores the answer after it exists. A leaked identifier has already left the building; no verdict recalls it.
Static rule lists	Fast, transparent, cheap to evaluate	Encode one organisation's policy at one moment. They go stale, and cannot express a policy document that changes.
RAG alone	Grounds answers in real source material	Has no view on which model should use that material, or whether the resulting answer may be delivered. Grounding is not governance.
Confidence scoring only	A usable signal where ground truth exists	Scores claims that have no ground truth, inventing certainty. The brief names this explicitly.
One global threshold	Simple to reason about and tune	Tuned tight it over-flags; tuned loose it underflags. It cannot be both, and different workflows need different settings.

THE GAP

Model selection and AI governance are usually separate systems that never exchange a signal. dotAI's differentiation is that they are one request-level control path.

4 · What makes dotAI different

Five capabilities, working on the same request. None of them is individually novel, and this document does not claim otherwise. The differentiation is that they inform one another.

CAPABILITY	WHAT IT CONTRIBUTES
1 · Capability-aware model selection	A model is eligible only if it can perform every atomic ability the request needs. A model missing one is excluded, not ranked lower.
2 · Cost-aware routing	Among eligible models, the cheapest that clears the request's requirement bar is recommended — decided per request, not per deployment.
3 · Risk-aware governance	Risks are judged together. Co-occurring findings escalate; a control may only tighten an outcome, never relax it.
4 · Policy-grounded verification	Decisions rest on the organisation's own uploaded policy, retrieved and cited, rather than a generic rule list.
5 · Accountable decisions	Every decision carries its risk, reason, confidence and supporting evidence, and a human can overturn it on the record.

Why the combination matters. A router that knows nothing about risk will happily select a cheap model for a request carrying sensitive data. A filter that knows nothing about capability will reject an answer that was poor only because the wrong model produced it. Joined, the same analysis that determines which model is capable also determines what governance applies — and both are recorded against the same request.

5 · System architecture



LAYER	ROLE
CAI	Reads the request and states what it requires. Runs on every query without exception.
Model Intelligence	Turns those requirements into an eligible set, then into three ranked choices.
Privacy firewall	Decides what may leave the system, before anything is sent.
Execution	Calls the selected model through OpenRouter using the caller's own key.
ControlPlane	Detects risk in the answer, retrieves policy evidence, verifies claims.
Decision	Places the result on a five-state ladder; controls may only tighten it.
Action Gate	Where the delivered answer carries an intent to act, gates that action separately before execution (\$12).
Audit + feedback	Records why, and captures human disagreement for measurement.

6 · CAI — query understanding

CAI is the runtime query analyser. It runs on `google/gemini-2.5-flash-lite`, chosen because it sits in front of every request and must therefore be fast and cheap. It classifies; it never writes the answer, and it never selects a model.

WHAT CAI DETERMINES, FOR EVERY QUERY

Input form	Text · Image · Document · Structured Data
Output form	Text · Image · Document · Structured Data · Vector
Sub-task	one of 16, e.g. "Document Intelligence", "Coding"
Required abilities	the specific atomic abilities this request needs (List A) — derived from the query, not its label

WHY CAI EXISTS

Model selection is only as good as the description of the work. A broad label is not a description: "conversation" is true of a greeting and of a request to compare two contracts, and those need very different models. CAI produces a description specific enough to route on.

NOT A "CONVERSATION" CLASSIFIER
An earlier build displayed a coarse routing bucket that labelled almost every text prompt Conversation. That was a real defect and is fixed: the classification shown, recorded and routed on is CAI's sub-task. There is one classifier on the request path, and no fast lane that skips it — including for a one-word greeting.

Validation. CAI's answer is not trusted blindly. The input form is taken from the attachments actually present rather than the model's reading of them; the output form is validated against the taxonomy and corrected by the required abilities where the two disagree; and any ability name outside the catalogue is discarded. If CAI is unreachable, a deterministic fallback classifies from the query's wording and labels itself as such.

7 · Model Intelligence

Model Intelligence answers one question: given what this request needs, which models can actually do it, and which of those should we use?

THE ELIGIBILITY RULE

List A the abilities this request requires (from CAI)
List B the abilities a model can perform (from the dataset)

A model is eligible only when $List\ A \subseteq List\ B$

A model missing even one required ability is excluded — not ranked lower. It is not a cheaper option; it is a model that will fail the request.

THE THREE RECOMMENDATIONS

TIER	DEFINITION
RECOMMENDED	The cheapest eligible model that clears the requirement bar this query sets
ALTERNATIVE	The cheapest eligible model strictly stronger than Recommended
BEST	The cheapest eligible model strictly stronger than Alternative

Intelligence rises strictly across the three. Where no model is a genuine step up, the tier is left empty rather than filled with a duplicate — three cards showing equivalent models would tell the user nothing.

WHY THIS BEATS THE ALTERNATIVES

Versus a leaderboard. A leaderboard ranks models globally. It cannot say that a model strong at reasoning is unable to read a document layout. Eligibility excludes; ranking merely orders.

Versus a fixed model. A fixed choice is either too expensive for simple work or too weak for hard work. It cannot be correct for both.

Versus always using the strongest. Measured in a controlled prototype test of 24 representative requests, routing this way cost 48% less than a fixed mid-tier model and 98% less than a fixed frontier model, with identical token assumptions in both arms. Prototype evaluation only — see §28.

VERIFIED CURRENT SCOPE

CURATED MODELS	PRIMARY MINI-TASKS	SUB-TASKS	CAPABILITY ROWS	RATED PAIRS
65	72	16	4,680 (2,240 positive)	410

8 · The primary mini-task system

Why broad labels are not enough. "Coding" covers writing a function, explaining a regular expression and translating between languages — different abilities that different models have to different degrees. Routing on the label sends all three to the same model. Routing on the abilities does not.

THREE LEVELS, AND HOW THEY RELATE

TERM	MEANING	EXAMPLE
Sub-task	A broad area of work, defined by an input and output form. 16 exist.	Document Intelligence
Primary mini-task	One atomic ability. It is not a parent of another ability. 72 exist.	Document Field Extraction
Capability set (List B)	Every mini-task one model is assessed as able to perform.	the 40 abilities model X covers

A sub-task is a grouping built from mini-tasks, not the other way round. A model is associated with a sub-task only when its capability set covers that sub-task's members — never by manual assignment.

```
request → List A { read a document's layout, pull named fields,
                    return structured JSON }

model    → List B { ...40 abilities this model covers... }

eligible ⇔ every member of List A appears in List B
```

9 · The curated model dataset

Model intelligence comes from one static file, `data/curated-model-intelligence.json`, which ships with the application. No live catalogue sync occurs at runtime.

WHY CURATED, AND WHY STATIC

- Stability.** The set of models a user sees does not shift between page loads because an upstream catalogue changed.
- Auditability.** A routing decision made today can be reproduced tomorrow against the same data.
- Correctness of comparison.** Cost and capability figures come from one source, so ranking them against each other is meaningful.

WHAT THE DATASET HOLDS

ELEMENT	REPRESENTATION
Model identity	Internal id plus the OpenRouter model id used for execution, so the model displayed is the model that runs
Pricing	Input and output price per million tokens, held separately — they commonly differ several-fold and a blended figure hides what a cost decision turns on
Intelligence	A 0-100 rating per sub-task. The same model is rated separately for coding and for document work.
Capability mapping	One row per model x mini-task, marked positive or negative, with the assessment evidence recorded alongside

LIMITATION — ANALYTICALLY ASSESSED CAPABILITIES

The capability labels are an expert analytic assessment recorded in the dataset, derived from model cards, published benchmarks and architecture. They were not produced by live-probing every model against every mini-task. The application carries this distinction through in code as `CAPABILITY_PROVENANCE = "ANALYTIC"` and must not present these labels as measured. A three-of-three probe harness is implemented (\$29) but has not been run at scale — doing so is on the order of 90,000 provider calls.

10 • ControlPlane governance

```
request → risk detection → overlapping-risk logic → policy + evidence
           → verification → decision → audit
```

THE DECISION LADDER

This is not a binary safe/unsafe classifier. An answer lands on one of five states, and a control may only move it up the ladder — never relax an outcome another control has already reached.

STATE	WHAT IT MEANS	WHEN IT IS USED
ALLOW	Delivered unchanged	No material finding. Most traffic.
ANNOTATE	Delivered with a caveat attached	A finding worth disclosing but not worth withholding the answer for — e.g. a claim that could not be fully supported.
REGENERATE	Answered again before delivery	The answer is correctable — the problem is with this attempt, not with the request.
HOLD	Routed to a person; not delivered yet	A human should look before this leaves. Consequential actions (see §12) and, under strict profiles, unresolved evidence.
BLOCK	Not delivered at all	A line the policy will not caveat past — e.g. critical identifiers heading outside the system.

OVERLAPPING RISKS

Findings are multi-label: one request can carry privacy, hallucination and high-consequence-action findings simultaneously. Counted separately these read as several mild flags. dotAI escalates on the intersection — a hallucinated answer that also triggers a consequential action is a different problem from either alone, and the ladder reflects that. This directly addresses the brief's overlapping-risk challenge.

UNCERTAINTY IS A STATE, NOT A SCORE

Where no evidence exists to check a claim against, the result is marked UNVERIFIABLE and escalated. It is not assigned a confidence number that was never earned. Under the Decision Support profile, unresolved evidence is itself disqualifying (§14).

COMPOUNDING RISK ACROSS A CONVERSATION

Session risk accumulates across turns. As it rises from low to high, verification depth rises with it — so a conversation that has been drifting toward sensitive ground is checked harder than its first turn was.

11 · Pre-generation privacy firewall

The problem. Every other control in the system inspects the model's answer. By the time an answer exists, anything sensitive in the request has already been sent to a third-party provider. No downstream verdict can recall it.

The response. The firewall inspects the request, before anything leaves.

user input → PII detection → policy check → decision → model

ALLOW	nothing sensitive found; sent unmodified
REDACT	values masked; the question still reaches the model
HOLD	not forwarded; routed to a person
BLOCK	refused outright; nothing is sent

Redaction is preferred wherever the policy permits it: a masked request still answers the user's question, and a blocked one does not. Attachment text is scanned as well as the prompt — a PDF can carry what the prompt does not.

WORKED EXAMPLE (FICTIONAL)

Request	"My account 4488-1234-5678 needs checking, can you help?"
Detected	account_number · critical · confidence 0.85
Policy	Baseline governance
Decision	REDACT
Model	"My account [REDACTED:ACCOUNT_NUMBER] needs checking, can you help?" — receives the question without the identifier — and still answers it.

Under the Decision Support profile the same request returns HOLD: a critical identifier is not forwarded to any model, and the request goes to a person instead. Same input, different governance — verified in a controlled run.

CONFIDENCE REPORTING

Detection confidence is taken from the weakest matching pattern, not an average. A decision is only as sound as its shakiest piece of evidence, and reporting one blended number for a checksum-backed card number and a loose address pattern would make the figure meaningless.

LIMITATION

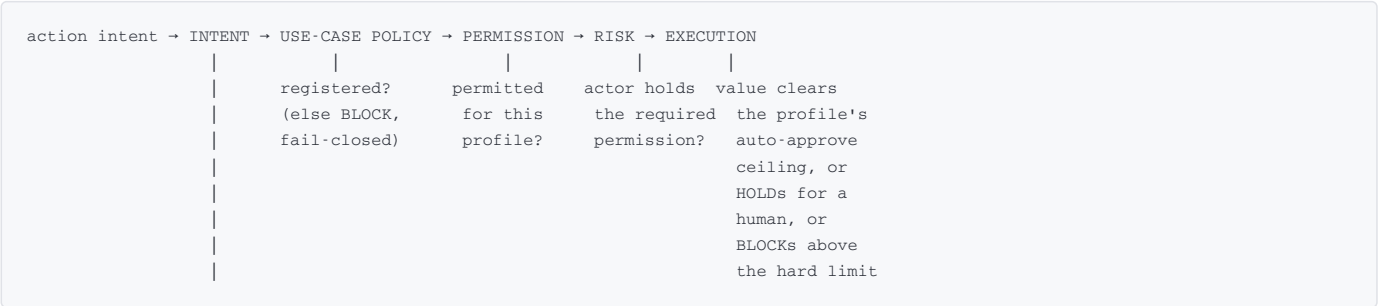
This is pattern-based detection over a defined class list. It is not complete privacy protection and must not be described as such. Data in an unusual format, or sensitive by context rather than by shape, may not be detected.

12 · Action Gate — governing consequential actions

NEW

The problem. Everything in §10 governs a piece of text — whether it may be shown to the user. Some answers don't just carry information; they carry an *intent to act*: issue a refund, approve a payment, send an email outside the organisation. Text-governance controls can clear the sentence and still miss that the sentence proposes doing something with real, sometimes irreversible, consequences. The brief's "high-consequence action" risk needs its own control, not another wording of the same one.

The response. Before any external action executes, it passes through the Action Gate — a five-stage check that runs alongside, not instead of, the answer-governance ladder in §10.



STAGE	WHAT IT CHECKS
Intent	Is this a registered action at all? Anything not in the policy registry is denied by default — fail-closed, not fail-open.
Use-case policy	Is this action even permitted under the governance profile in effect? A profile's blocked-action list always wins.
Permission	Does the actor hold the specific permission the action requires (e.g. <code>refunds.write</code>)? No permission, no action, regardless of value.
Risk	Does the action's value clear the profile's auto-approve ceiling? Below it, it proceeds; above it, it HOLDS for a human; above a hard limit, it BLOCKs outright no matter who is asking.
Execution	Only an action that has cleared every prior stage runs. A human's approve / reject / edit / regenerate decision on a held action is logged the same way the answer-governance overrides are in §15.

A profile's escalation threshold may only be *stricter* than an action's own registered limit, never looser — the same tighten-only rule that governs the decision ladder in §10 applies here too. This is why a HOLD for "consequential actions" in §10's table is not just a label: it is this gate actually firing.

WHY A SEPARATE GATE

Answer-governance asks "may this text reach the user?" Action governance asks a different question: "may this consequence occur?" A refund proposal that reads perfectly well as text can still be the wrong action to let execute unattended. Keeping the two checks separate means an action can be held even when the answer that proposed it was otherwise clean.

Implemented in `lib/action-gate/` , exposed through the review queue in the control panel, and covered by its own test file — see §25 and §26.

13 · Policy RAG and evidence

The problem. A static rule list encodes one organisation's policy at one moment. It cannot express a policy document, and it goes stale as expectations change — which the brief names directly.

```
upload → extract → chunk + index → retrieve → cite → decision
```

STAGE	BEHAVIOUR
Upload	PDF, DOCX, TXT and Markdown are supported.
Extract	The text layer is read. A scanned file with no text layer is rejected, not stored as an empty document — a silently empty policy is worse than none.
Chunk and index	Split into sections and embedded for semantic search.
Retrieve	Only sections above a relevance floor are returned. Retrieval may return nothing.
Cite	The verdict names the sections it rested on, and those citations are stored with the decision.
Delete	Deletion is permanent and removes the document from retrieval immediately. Decisions already made keep the evidence they recorded.

WHY A RELEVANCE FLOOR MATTERS

A top-K retrieval that always returns K results will always produce a citation, including when nothing relevant exists — which is a citation that misleads. The floor means the honest answer "no applicable policy was found" is available, and when it occurs the claim becomes UNVERIFIABLE and the governance layer escalates rather than asserting.

RETRIEVAL OFF DOES NOT MEAN GOVERNANCE OFF

Disabling retrieval for a request does not disable the checks. With no evidence to verify against, claims become unverifiable — which the policy then escalates. Turning off the evidence makes the system more cautious, not less.

14 · Governance profiles

Four profiles, resolved from deployment configuration. They are governance policy, not model-selection options — they never appear as a choice in the chat interface, because how strictly a request is judged should not depend on a dropdown the user has no basis for setting.

Critically, these are not four sets of thresholds under different names. Each runs a different sequence of controls.

PROFILE	CONTROL SEQUENCE	WHAT IT TRADES
Baseline — general purpose	detect → verify → decide → deliver	Verification light, raised by risk. No special obligation to retrieve or cite.
Customer Support — latency-first	detect → privacy → policy → fast verification → decide	Verification is capped to stay inside a 1,200 ms budget. Customer data is not traded away: privacy findings block outright.
Internal Copilot — grounded	detect → internal data → retrieve → access → verify → cite	Retrieval is forced on regardless of the request setting; an uncited answer is a workflow failure, not a style preference.
Decision Support — consequential	detect → retrieve → verify → uncertainty → strict policy → human approval	Deep verification. Unresolved evidence is disqualifying. Any material sum needs a person.

FOUR BEHAVIOURAL DIFFERENCES, NOT NUMERIC ONES

RETRIEVAL POLICY	CITATION OBLIGATION	VERIFICATION BOUNDING	UNCERTAINTY RULE
forced vs on-demand	required vs optional	capped vs run to completion	blocking vs annotating

Verified. Given the same contradicted claim, three profiles return HOLD and Decision Support returns BLOCK. Given the same request containing an account identifier, Baseline returns REDACT and Decision Support returns HOLD. Same input, different governance.

This is dotAI's direct answer to the brief's requirement that different use cases carry different risk tolerance and latency budgets.

15 · Feedback, metrics and thresholds

decision → human review → override recorded → agreement measured → threshold reviewed

A control that blocks is only useful if you know what it blocks wrongly. Human verdicts are persisted with the reviewer's reason and, where supplied, a corrected output. Agreement rate is computed overall and per decision type, so the system can be told it was wrong on the record.

THRESHOLD SELECTION

The firewall's operating threshold was chosen from a labelled set rather than picked. The full sweep:

THRESHOLD	FALSE POSITIVES	FALSE NEGATIVES	PRECISION	RECALL
0.50 – 0.85	0	0	1.00	1.00
0.90	0	3	1.00	0.50
0.95	0	4	1.00	0.33
0.99	0	5	1.00	0.17

0.85 was chosen — the highest threshold that still produces zero false negatives. The reasoning is recorded in the implementation: the two errors are not equally costly. A false positive costs a user a rephrase; a false negative sends sensitive data to a third party and cannot be undone. Recall is therefore treated as a constraint rather than a term traded against precision, and among thresholds that miss nothing, the highest raises the fewest false alarms.

LIMITATION — SMALL EVALUATION SET

These figures come from a 14-case hand-labelled set (8 should-pass, 6 should-stop), deliberately small enough for a person to check every case. It is a regression fixture for the decision rule, not a benchmark of detector accuracy on live traffic, and must not be presented as one.

16 · Auditability and the decision trace

Every governed request records a trace answering the question an auditor actually asks: why did the system do that?

RISK	what was detected, by class, with counts
DECISION	which of the five states was reached
CONFIDENCE	how strong the weakest supporting signal was
REASON	in plain language, why this rather than something else
EVIDENCE	which policy sections the verdict rested on
OVERRIDE	the human verdict, where one was given

Alongside these, each event stores the CAI classification, the required abilities, the eligible model count, the model actually executed, the governance workflow that ran, token counts, cost and latency. Because the classification recorded is the same one shown to the user, the audit record and the interface cannot disagree.

Why it matters. Foundation models reached through an API cannot be inspected internally — the brief names this. Governance therefore has to be demonstrable at the boundary: what was sent, what came back, what was decided, and on what grounds. A decision a reviewer cannot reconstruct is one they can only accept or ignore.

17 • Multimodal chat

FORM	INPUT	OUTPUT	NOTES
Text	yes	yes	The default path.
Image	yes	yes	Generated images render inline and download as real files.
Document	yes	yes	PDF, DOCX, TXT, MD in. DOCX rendered out by dotAI from model text.
Structured data	yes	yes	JSON and tabular content, produced as text and rendered.

Governed identically. An uploaded PDF is scanned by the privacy firewall before its text reaches any model. A generated image is produced only after the decision permits it — so a blocked request leaves no file behind. Documents and structured data are rendered by dotAI after the governance decision, which is what makes that guarantee possible.

Downloads. Generated images and documents download as real files rather than previews. Image download fetches to a blob rather than relying on an anchor, because generated images arrive as data URLs that some browsers refuse to download directly.

NOT IMPLEMENTED

Audio and video input/output are not implemented. Voice transcription exists in the codebase but requires an account privacy setting that permits the provider's transcription endpoint; it is not part of the demonstrated scope.

18 · End-to-end worked example

One request, through every layer. This is a real run, not an illustration.

"Read this invoice PDF, extract invoice number, date, tax and total, and return them as JSON."

#	STAGE	RESULT
1	CAI classifies	Document → Text → Document Intelligence (gemini-2.5-flash-lite)
2	Required abilities (List A)	read a document's layout · pull named fields · return structured JSON
3	Eligibility (List A ⊆ List B)	16 of 65 models eligible; 49 excluded for a missing ability
4	Recommended	Gemini 2.5 Flash Lite — \$0.10 in / \$0.40 out, intelligence 74 (cheapest eligible)
5	Alternative	Claude Opus 5 — intelligence 93
6	Best	Gemini 3.1 Pro — intelligence 94
7	Privacy firewall	Document text scanned before sending; nothing sensitive found → ALLOW
8	Execution	Recommended model called via OpenRouter
9	Result	INV-8842 · 12 May 2026 · Northwind Ltd · 1000.00, returned as JSON
10	Verification	Claims checked against the source document
11	Decision	ANNOTATE — delivered with the caveat recorded
12	Audit	risk → decision → confidence → reason → evidence, all stored

The model named at step 4 is the model executed at step 8. Where a provider is unreachable, execution falls back to the next eligible model, the substitution is announced, and the interface updates to show what actually ran — so the displayed model and the executed model never diverge silently.

19 • Key features

FEATURE	WHAT IT DOES	WHY IT MATTERS
CAI query understanding	Classifies input, output, sub-task and required abilities on every request	Model choice is only as good as the description of the work
Capability matching	List A $\subseteq$ List B eligibility	A model that cannot do part of the job is excluded, not ranked lower
Three-tier selection	Cheapest capable, a strict step up, the strongest	Gives a real choice instead of one opaque pick
Privacy firewall	Detects and acts on sensitive data before generation	A leak cannot be undone by a later verdict
Policy RAG	Retrieves and cites the organisation's own policy	Decisions follow the document, which changes; rules do not
Multi-label risk + intersections	Co-occurring findings escalate together	Two mild flags can be one serious problem
Five-state decision ladder	Allow, annotate, regenerate, hold, block	Proportionate response instead of a binary filter
Action Gate	Refunds, payments and outbound sends pass a fail-closed permission-and-value gate, separate from answer governance, before they execute	An answer can be governed; an action needs its own control (§12)
Governance profiles	Four distinct control sequences	Different workflows genuinely need different controls
Decision trace	Risk, decision, confidence, reason, evidence	A decision nobody can reconstruct cannot be governed
Feedback + thresholds	Overrides captured; FP/FN measured	The system can be told it is wrong, and measured over time
Multimodal I/O	Text, image, document, structured data	Enterprise work does not arrive as plain text
Prompt Library	Reusable prompts with typed variables	A vague prompt wastes even a strong model

20 · Technology stack

LAYER	TECHNOLOGY	ROLE
Framework	Next.js 15.1 (App Router)	Server components, API routes, streaming responses
Language	TypeScript 5.7	Strict typing across application and tests
UI	React 19, Tailwind CSS 3.4, Radix UI, lucide-react	Interface, accessible primitives, icons
Database	SQLite via Prisma 6.19	41 models covering events, policy, feedback, conversations
Model access	OpenRouter	Single gateway to all executed models
Query analysis	google/gemini-2.5-flash-lite	CAI classifier — fast and cheap, runs per request
Embeddings	@xenova/transformers	Local semantic embeddings for policy retrieval
Documents	pdfjs-dist, mammoth, docx	PDF and DOCX extraction; DOCX generation
Validation	Zod 3.24	Runtime schema validation on every API boundary
Testing	Vitest 2.1	53 test files, 694 tests

Why SQLite. The prototype must run from a clone with no external services. Prisma keeps the schema portable, so moving to Postgres is a connection-string change rather than a rewrite.

Why local embeddings. Policy documents are the most sensitive material in the system. Embedding them locally means an uploaded policy is not sent to a third party merely to be indexed.

21 · Installation

PREREQUISITES

- Node.js 20 or later and npm
- An OpenRouter API key — optional for a first look; without one dotAI runs in mock mode and executes no models

SETUP

```
# 1 · Install dependencies. Prisma Client is generated automatically.
npm install

# 2 · Create the environment file
cp .env.example .env

# 3 · Create the database
npm run db:push

# 4 · Start the development server
npm run dev
```

Open `http://localhost:3000` . The application runs immediately in mock mode.

AVAILABLE SCRIPTS

COMMAND	PURPOSE
npm run dev	Development server; pushes the schema first
npm run build	Production build; generates Prisma Client and pushes the schema first
npm start	Serve the production build
npm test	Full test suite (Vitest)
npm run typecheck	TypeScript, no emit
npm run lint	ESLint via next lint
npm run db:push	Apply the Prisma schema to the database

FIRST RUN

The first request after a cold start is slower: the local embedding model is downloaded and cached, and the policy corpus is indexed. Subsequent requests are unaffected.

22 · Configuration

All configuration lives in `.env`. The committed `.env.example` lists every variable with its default.

CORE

VARIABLE	DEFAULT	MEANING
DATABASE_URL	file:./dev.db	Prisma connection string
MOCK_MODE	true	When true, no model is executed and outputs are labelled simulated
OPENROUTER_API_KEY	empty	Your key. Can also be connected in Settings without editing this file.

ROUTING AND ANALYSIS

VARIABLE	DEFAULT	MEANING
CAI_MODEL	google/gemini-2.5-flash-lite	The query analyser. Classifies only; never writes answers.
CAI_MAX_COST_USD	0.002	Per-request ceiling for analysis
CAI_TIMEOUT_MS	20000	Analysis is abandoned past this; a deterministic fallback classifies instead
GOVERNANCE_PROFILE	BASELINE	Which governance profile the deployment runs (\$14)
RELIABILITY_MIN_SAMPLE	5	Observed runs before learned reliability influences anything
AUTO_MODE_DEFAULT	true	Run the recommended model without asking

CONNECTING A KEY WITHOUT EDITING FILES

Settings → Connect OpenRouter accepts a key at runtime and shows a masked confirmation. Without a key the interface states plainly that dotAI is in mock mode rather than presenting simulated output as real.

23 · How to use dotAI

SECTION	WHAT YOU DO THERE
Chat	Send a request. See the three recommended models with their costs and ratings, choose one or let auto mode run the recommendation, and open any answer's detail to see the decision and its evidence.
Models	Browse Model Intelligence by input form, output form and task. Every model shows its input cost, output cost and rating for that task.
Library	Save reusable prompts with typed variables. Fill them in, choose an existing conversation or a new one, and the prompt lands in the composer ready to send — never sent automatically.
Policy & Audit	Upload the policy documents decisions should rest on. Review the audit trail of past decisions with their evidence.
Feedback	Metrics (decision quality), Control (why individual decisions were made, including Action Gate holds), and Usage (spend by model and decision outcome).
History	Past conversations, resumable.
Settings	Connect your OpenRouter key; set cost preference and verification depth.

A TYPICAL FIRST SESSION

- 1. Connect your key in Settings.
- 2. Upload a policy document under Policy & Audit.
- 3. Ask a simple question in Chat — note the inexpensive model recommended.
- 4. Ask a demanding one, with documents attached — note that the recommendation changes and the eligible pool shrinks.
- 5. Open the decision detail to see the risk, reason and evidence.
- 6. Check Feedback → Usage to see what was actually spent.

24 • Example requests

REQUEST	CLASSIFIED AS	WHAT YOU SHOULD SEE
"Hi, how are you?"	Text → Text → General Chat & Writing	~51 of 65 models eligible; an inexpensive model recommended
"Write a Python function to calculate factorial."	Text → Text → Coding	~42 eligible; a coding-capable model recommended
"Compare these two vendor contracts and explain which carries more risk." (both attached)	Document → Text → Document Intelligence	~14 eligible; a markedly stronger model recommended
"Extract the invoice number and total from this document."	Document → Text → Document Intelligence	~16 eligible; cheapest capable model recommended
"Generate an image of a golden retriever."	Text → Image → Image Generation	~5 eligible; a real downloadable image produced
"My account 4488-1234-5678 needs checking."	Text → Text → General Chat & Writing	Firewall returns REDACT; the model receives the question without the identifier

Eligible counts are from the shipped dataset and will change if the dataset does. The point is the shape: harder and more specific requests have smaller eligible pools.

25 · Repository structure

```
dotai/
├─ app/                                Next.js App Router
│  └─ chat/                            conversation interface
│  └─ models/                          Model Intelligence browser
│  └─ library/                         prompt library
│  └─ policy/                          policy upload and audit trail
│  └─ metrics/ control/ usage/          grouped under Feedback in the sidebar
│  └─ history/ settings/
│  └─ api/                             19 route groups (chat, policy, intelligence,
│                                     governance, action, library, usage, audit, ...)
│
├─ lib/                                application logic
│  └─ controlplane.ts                  the governed request path
│  └─ cai/                            query analyser configuration
│  └─ intelligence/                   curated dataset, capability routing, tiers
│  └─ governance/                    profiles, decisions, privacy firewall,
│                                     threshold evaluation
│  └─ action-gate/                   consequential-action gating — intent, policy,
│                                     permission, risk, execution (§12)
│  └─ policy/ rag/ retrieval/          policy ingestion, chunking, retrieval
│  └─ verification/ performance/ responsibility/ answer checking
│  └─ providers/                     OpenRouter integration
│  └─ documents/                     generation/ extraction and rendering
│  └─ audit/ learning/ cost/          trace, feedback signal, accounting
│  └─ library/ models/ routing/
│
├─ components/                        React components by area
├─ data/
│  └─ curated-model-intelligence.json  the model universe (§9)
├─ prisma/schema.prisma              41 models
├─ tests/                            53 files, 694 tests
├─ docs/                             supplementary notes
└─ types/                            shared TypeScript types
```

26 · Testing and verification

All four pass on the shipped code.

```
npm test          # 694 tests across 53 files
npm run typecheck # TypeScript, no emit
npm run lint      # ESLint
npm run build     # production build
```

WHAT THE TESTS COVER

AREA	EXAMPLES OF WHAT IS ASSERTED
Capability routing	Eligible models satisfy List A ⊆ List B; tiers rise strictly in intelligence; every rejection names the missing ability
Classification	CAI is the only classifier; no query is silently labelled "conversation"; List A varies with the query
Privacy firewall	All four outcomes; profile-dependent divergence; attachment text is scanned; critical classes mirror the detector's own severities
Governance	Four distinct stage sequences; a control may only tighten; intersection escalation; uncertainty handling
Action gating	Unregistered actions denied by default; permission checks block without the right grant; value above the hard limit blocks regardless of permission; high-value actions hold for human approval; profile-specific allow/block lists enforced
Threshold evaluation	FP/FN counted, precision and recall derived, and the chosen point has zero false negatives
Policy and RAG	Extraction, chunking, relevance floor, citation building, permanent deletion
Multimodal	Every supported input/output pair, including that a blocked request leaves no file behind
Model dataset	Costs and ratings are populated for every listed model; no duplicate models; eligible sets reproduce the dataset's own worked examples

A NOTE ON RUNNING TESTS

Some tests build the model catalogue on first run and may report failures on a cold database. Run `npm test` a second time — the suite is green on a warm database. Tests that assert no-credential behaviour will fail if `OPENROUTER_API_KEY` is set; leave it empty when running the suite.

27 · Security and secrets

- Your key stays yours.** Models are called with the key you supply, billed to your account. Nothing is proxied through a dotAI service.
- The key is never committed.** `.env` is gitignored and `.env.example` ships with an empty value. The Settings screen shows only a masked confirmation.
- Policy documents are embedded locally.** An uploaded policy is not sent to a third party to be indexed.
- Redaction happens before transmission.** Where the firewall redacts, the provider receives the masked text — the original never leaves.
- Deletion is real.** Deleting a policy document removes it from retrieval immediately.

PROTOTYPE-LEVEL AUTHENTICATION

User identity is a signed cookie. This is sufficient for a local prototype and is not production authentication. Any real deployment needs proper identity and access control (\$29).

28 · Scope and limitations

Stated plainly, because a governance product that overstates itself has undermined its own argument.

AREA	STATUS
Capability labels	ANALYTIC. An expert assessment recorded in the dataset — not live probes of every model against every ability. The probe harness exists but has not been run at scale.
Cost figures	PROTOTYPE, GROSS NOT NET. The 48% and 98% reductions come from a controlled 24-request test with identical token assumptions in both arms, and they are gross token savings. They do not net out CAI classification overhead (charged on every query, including ones later blocked), policy-RAG hosting and embedding cost, or human-review labor on HOLD-queue items — those are illustrative planning figures, not measured production costs. Real net ROI depends on a traffic mix, threshold tuning, and reviewer throughput not yet measured.
FP/FN metrics	SMALL SET. A 14-case hand-labelled regression fixture, not a live-traffic accuracy benchmark.
Model universe	65 curated models. Not the full provider catalogue.
Authentication	Signed cookie. Not production identity.
Audio and video	Not implemented.
Latency impact	Not measured. No figure is claimed.
Manual review reduction	Not measured. No figure is claimed.
Workflow stage tracing	Governance stage names declare each profile's sequence and drive its behaviour, but each named stage does not emit its own separate trace event.
Scale	SQLite, single instance. Not load-tested.

CLAIMS DELIBERATELY NOT MADE

- No production readiness or regulatory compliance claim.
- No enterprise ROI, customer count, deployment or pilot.
- No accuracy or hallucination-reduction percentage.
- No benchmark comparison against a named competitor.
- No claim that privacy protection is complete — it is pattern-based over a defined class list.

29 · Roadmap

NEXT	WHY IT MATTERS
Execute the capability probe matrix	Converts analytic capability labels into measured ones. The harness — three isolated examples per ability, all-or-nothing labelling, per-example evidence, resumable and cost-capped — is implemented but unrun at scale.
Broaden the model universe	Beyond the curated 65, with the same rating discipline.
Per-stage execution tracing	Each governance stage emitting its own trace event, rather than declaring the sequence.
Enterprise identity and access	Replace the prototype cookie with real authentication and role-based access.
Continuous evaluation on live traffic	Replace the fixed 14-case set with a growing labelled corpus drawn from real overrides.
Cost telemetry at production mix	Turn the prototype cost comparison into a measured operational figure, netting out CAI, RAG hosting and review labor as described in §28.
Richer policy libraries	Multi-jurisdiction packs and versioned policy history.
Broaden the action registry	The Action Gate (§12) currently governs a small fixed set of action types; extending its registry follows the same discipline as the model dataset — deny by default until an action is explicitly added.

30 · Summary

dotAI is a working prototype of a control layer for enterprise AI. For every request it determines what the request actually requires, which models are capable of delivering it, what information may safely be sent, whether the answer stands up to evidence, and what should happen next — recording the reasoning at each step.

The differentiation is not any single capability. It is that model selection and AI governance, normally separate systems that never exchange a signal, are joined into one request-level path. The same analysis that establishes which models can do the work also establishes what governance applies to it.

VERIFIED VALUE

Automated tests passing	694 across 53 files
Typecheck, lint, production build	All clean
Curated models · mini-tasks · sub-tasks	65 · 72 · 16
Governance profiles	4, with distinct control sequences
Decision states	5 — allow, annotate, regenerate, hold, block
Action Gate stages	5 — intent, use-case policy, permission, risk, execution

IN ONE LINE

An assistant answers the request. A control layer decides what is allowed to happen.